

MODULE 9 · TROUBLESHOOTING

Logs, Events & Metrics

What the app said, what Kubernetes did, what it's actually using

WHY IT MATTERS

Every triage in this module ends here

Across Module 9 you diagnosed failures with `describe` and `get`. Logs, events, and metrics are the raw evidence underneath those commands — what a process printed, the timeline of what the cluster actually did, and the live resource numbers.

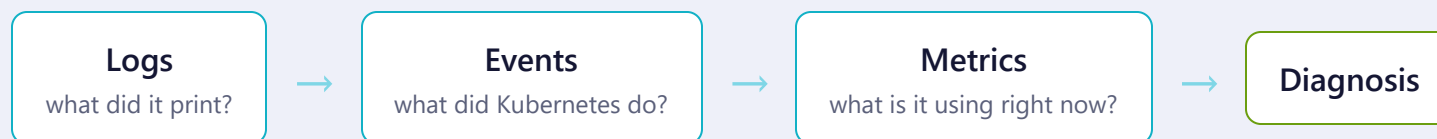
They don't diagnose anything by themselves. They're the sources you read — and they're also the three commands you'll reach for constantly once nothing is broken.

BY THE END YOU CAN

- Pull logs from a crashed or multi-container Pod
- Trace a failure through the event timeline
- Compare live usage against requests and limits

CONCEPT

Three questions, three commands



None of these three change a single resource — they're pure observation. And they compose with everything you've already learned: `-n`, `-l`, `-o jsonpath`, `describe`.

kubectl logs: one container to a fleet

NEED	COMMAND	NOTE
Single container	<code>kubectl logs POD</code>	Only choice when the Pod has one container
One of several	<code>kubectl logs POD -c NAME</code>	Without <code>-c</code> , kubectl defaults to the first container and warns
Crashed / restarted	<code>kubectl logs POD --previous</code>	The last terminated instance — not the fresh, possibly-empty one
Every container in a Pod	<code>kubectl logs POD --all-containers</code>	Interleaves every stream
Every Pod matching a label	<code>kubectl logs -l app=foo --prefix</code>	Tags each line <code>[pod/name/container]</code>

MENTAL MODEL

Logs are **per-container**, not per-Pod — a Pod with two containers has two independent log streams, and kubectl needs to know which one you want.

The container name decides which stream you get

containers:

- name: **nginx** # logs POD -c nginx
image: **nginx:1.27**
- name: **sidecar** # logs POD -c sidecar
image: **busybox:1.36**

A crashing container is the same idea, one step removed: the running instance may not have printed anything yet, but the one before it did.

```
$ kubectl logs log-multi
Defaulted container "nginx" out of: nginx, sidecar

$ kubectl logs log-crash
# (nothing yet – fresh instance)

$ kubectl logs log-crash --previous
crash attempt 4, exiting...
```

Events name what the cluster did, not the app

WARNING REASON	MEANING
FailedScheduling	No suitable node found
Failed (pull)	Image pull failed — check name / tag / registry access
BackOff	Container keeps crashing — check Last State and <code>logs --previous</code>
Unhealthy	A probe failed
FailedMount	Volume mount failed — check PV / PVC / StorageClass

RETENTION IS SHORT

Events expire from etcd (default ~1 hour). Check them while the failure is still fresh — they won't be there tomorrow.

describe pod → Events is always the first look

The Events section at the bottom of `describe pod` is a timeline for that one object: Scheduled → Pulling → Failed for a bad image tag. A scheduling failure follows the exact same pattern, just with `FailedScheduling` instead.

`get events` widens that same view — sorted, cluster-wide, or filtered to just the Warnings.

```
$ kubectl get events --sort-by=.metadata.creationTimestamp
$ kubectl get events --field-selector type=Warning
$ kubectl describe pod event-debug-pod
Events:
  Normal    Scheduled    ...
  Normal    Pulling     ...
  Warning   Failed      Failed to pull image
```

top reads live numbers, not what you asked for

`kubectl top` shows real-time CPU/memory usage, sourced from the **Metrics Server** add-on — a separate component from the API server. No Metrics Server, no `top`, full stop.

The number only means something next to **requests** and **limits**: far below requests is over-provisioned; close to limits risks throttling or an OOMKilled.

CONFIRM IT'S RUNNING

`kubectl top nodes` returning data is the check — an error means the pipeline isn't there.

```
$ kubectl top nodes
$ kubectl top pods -n training --sort-by=cpu
$ kubectl top pods -n training --containers
$ kubectl get pods -o custom-columns=\
NAME:.metadata.name,CPU_REQ:.spec.containers[0].resources.requests.cpu
```


Where people trip

- **Relying on "first container."** Multi-container logs without `-c` grab whichever container is listed first — fragile, not a habit.
- **Reading the live logs on a crash.** A cycling `CrashLoopBackOff`'s current instance may have printed nothing — `--previous` has the story.
- **Chasing app logs on a scheduling failure.** `FailedScheduling` and image-pull failures live in events, never in logs.
- **Trusting top without checking Metrics Server first.** An error isn't "zero usage" — the pipeline isn't running.

Commands to keep at hand

```
$ kubectl logs POD [-c CONTAINER] [--previous]          # what the process said
$ kubectl logs -l app=foo --prefix --all-containers    # every matching Pod, every container
$ kubectl get events --sort-by=.metadata.creationTimestamp
$ kubectl get events --field-selector type=Warning
$ kubectl describe pod NAME                            # Events for one object
$ kubectl top nodes
$ kubectl top pods -n NS --sort-by=cpu --containers
```

Tip: logs say what the app did, events say what Kubernetes did, metrics say what's actually being used — three different questions, three different commands.

RECAP

Logs, events, metrics — and the whole toolkit

- Logs are per-container — use `-c`, `--previous`, and label selectors
- Events are the cluster's own timeline — `get events --sort-by, describe` for one object
- Metrics need Metrics Server — read usage next to requests/limits, never alone
- `describe` → `logs` → `events` → `top`: the same triage instinct from Lab 9.1, now complete

HANDS-ON

Now run Lab 9.11
[labs/9.11/](#)

Course complete. Build, run, connect, store, scale, secure, package, diagnose — the full loop.